

TP: Volumes Docker	
Module : DevOps Filière : Génie Informatique Par : Pr. ALAMI Nabil	Année Universitaire : 2025/2026

1. Contexte

Dans une architecture conteneurisée, les conteneurs sont faciles à créer, arrêter, supprimer et recréer. Cette souplesse pose cependant un problème important : les données produites dans un conteneur ne sont pas forcément persistantes.

Dans ce TP, vous allez étudier la manière dont Docker gère la persistance à l'aide des volumes. Vous allez manipuler des volumes nommés, les monter dans des conteneurs, les inspecter, les partager entre plusieurs conteneurs et les utiliser dans un cas réel avec une base de données. L'objectif est de comprendre non seulement comment utiliser un volume, mais surtout pourquoi il est indispensable dans de nombreux contextes DevOps.

2. Objectifs

À l'issue de ce TP, vous devez être capables de :

- Expliquer pourquoi le système de fichiers d'un conteneur ne suffit pas pour la persistance
- Créer et supprimer des volumes Docker
- Lister et inspecter les volumes existants
- Monter un volume dans un conteneur
- Vérifier la persistance des données après suppression d'un conteneur
- Partager un volume entre plusieurs conteneurs
- Utiliser un volume avec un service d'état, notamment MySQL
- Comparer un volume Docker avec le stockage interne d'un conteneur
- Comparer la syntaxe `-v` et la syntaxe `--mount`
- Distinguer volume nommé et bind mount
- Diagnostiquer des problèmes liés aux volumes
- Appliquer de bonnes pratiques de gestion du stockage dans Docker

3. Travail demandé

Vous devez :

- Réaliser toutes les manipulations demandées

- Relever précisément les commandes utilisées
- Comparer les comportements observés
- Répondre aux questions d'analyse
- Résoudre les cas de debug proposés
- Produire un rapport structuré et argumenté

4. Ressources de travail

Images recommandées :

- alpine:latest
- nginx:latest
- mysql:latest

Vous pourrez aussi utiliser, si nécessaire une image Node.js ou une image Spring Boot déjà construite.

5. Mise en évidence du problème de persistance

5.1 Objectif

Montrer qu'un conteneur seul ne constitue pas une solution de stockage persistant.

5.2 Manipulation 1 — Création de données sans volume

1. Lancez un conteneur léger en mode interactif.
2. Créez à l'intérieur du conteneur un répertoire de test.
3. Créez un ou plusieurs fichiers texte dans ce répertoire.
4. Vérifiez leur présence.
5. Quittez le conteneur.
6. Supprimez ce conteneur.
7. Recréez un nouveau conteneur à partir de la même image.
8. Vérifiez si les fichiers créés précédemment existent encore.

Les que vous devez observer :

- Comportement des données après suppression du conteneur.
- Différence entre image et instance de conteneur.
- Caractère éphémère du système de fichiers interne du conteneur.

Questions :

1. Pourquoi les données créées dans un conteneur disparaissent-elles après suppression de celui-ci ?
2. Pourquoi ce comportement est-il problématique pour certaines applications ?

3. Quels types d'applications peuvent tolérer ce fonctionnement, et lesquels ne le peuvent pas ?

6. Création et gestion des volumes Docker

6.1 Objectif

Apprendre à créer, lister, inspecter et supprimer des volumes nommés.

6.2 Manipulation 2 — Création d'un volume nommé

1. Créez un volume Docker nommé.
2. Listez tous les volumes présents sur votre machine.
3. Identifiez le volume que vous venez de créer.
4. Inspectez ce volume.
5. Relevez dans le rapport :
 - Son nom
 - Son driver
 - Son point de montage

Questions :

1. Qu'est-ce qu'un volume Docker nommé ?
2. Pourquoi dit-on qu'un volume a son propre cycle de vie ?
3. En quoi un volume est-il différent d'un simple fichier interne à un conteneur ?

6.3 Manipulation 3 — Suppression d'un volume

1. Créez un second volume de test.
2. Listez les volumes présents.
3. Supprimez ce volume.
4. Vérifiez sa disparition.
5. Tentez ensuite de supprimer un volume encore utilisé par un conteneur.
6. Observez le comportement de Docker.
7. Expliquez le résultat obtenu.

Ce que vous devez constater :

- Les données persistent indépendamment du conteneur
- Le volume peut être réutilisé par plusieurs conteneurs successifs

Questions :

1. Pourquoi Docker empêche-t-il certaines suppressions de volume ?
2. Quelle différence entre un volume inutilisé et un volume attaché à un conteneur ?

3. Pourquoi le nettoyage des volumes doit-il être réalisé avec prudence ?

7. Montage d'un volume dans un conteneur

7.1 Objectif

Comprendre comment associer un volume à un conteneur et comment ce volume permet la persistance des données.

7.2 Manipulation 4 — Montage d'un volume nommé dans un conteneur

1. Créez un volume nommé.
2. Lancez un conteneur en montant ce volume sur un répertoire de votre choix.
3. Entrez dans le conteneur.
4. Créez plusieurs fichiers dans le répertoire monté.
5. Sortez du conteneur.
6. Arrêtez puis supprimez le conteneur.
7. Relancez un nouveau conteneur en montant le même volume au même emplacement.
8. Vérifiez la présence des fichiers créés précédemment.

Questions :

1. Pourquoi les fichiers sont-ils toujours présents après suppression du premier conteneur ?
2. Où se trouvent réellement les données stockées ?
3. Pourquoi ce mécanisme est-il essentiel pour les applications avec données persistantes ?
4. Dans quelle situation un stockage éphémère peut-il malgré tout être acceptable ?

8. Utilisation d'un volume avec plusieurs conteneurs

8.1 Objectif

Montrer qu'un même volume peut être partagé entre plusieurs conteneurs.

8.2 Manipulation 5 — Partage de données entre conteneurs

Travail demandé

1. Créez un volume nommé.
2. Lancez un premier conteneur en montant ce volume.
3. Créez un fichier dans le volume.
4. Lancez un second conteneur en montant ce même volume.
5. Vérifiez si le second conteneur voit les mêmes fichiers.
6. Modifiez ou ajoutez des fichiers depuis le second conteneur.

7. Retournez dans le premier conteneur et vérifiez la visibilité des changements.

Expliquez :

- Comment plusieurs conteneurs peuvent partager le même volume.
- Quels sont les avantages de ce mécanisme.
- Quels risques peuvent apparaître si plusieurs conteneurs modifient les mêmes données.

Questions

1. Pourquoi le partage d'un volume peut-il être utile dans une architecture applicative ?
2. Dans quels cas faut-il être prudent avec un volume partagé en écriture ?
3. Ce mécanisme convient-il de la même manière à tous les types d'applications ?

9. Utilisation des volumes avec MySQL

9.1 Objectif

Mettre en œuvre un cas réaliste de persistance avec une base de données conteneurisée.

9.2 Manipulation 6 — MySQL sans volume

Travail demandé

1. Lancez un conteneur MySQL sans volume.
2. Créez une base de test.
3. Créez une table simple.
4. Insérez quelques données.
5. Vérifiez que les données existent.
6. Arrêtez puis supprimez le conteneur.
7. Relancez un nouveau conteneur MySQL sans volume.
8. Vérifiez si la base et les données existent encore.

9.3 Manipulation 7 — MySQL avec volume

Travail demandé :

1. Créez un volume nommé dédié à MySQL.
2. Lancez un conteneur MySQL avec :
 - Le volume monté sur le répertoire de données de MySQL
 - Les variables d'environnement nécessaires
3. Créez une base de test.
4. Créez une table.

5. Insérez quelques lignes.
6. Vérifiez la présence des données.
7. Arrêtez puis supprimez le conteneur.
8. Recréez un nouveau conteneur MySQL en réutilisant le même volume.
9. Vérifiez si les données sont toujours présentes.

10. Comparaison -v et --mount

10.1 Objectif

Comparer les deux syntaxes principales utilisées pour monter un volume dans un conteneur.

10.2 Manipulation 8 — Montage avec -v

1. Montez un volume nommé dans un conteneur en utilisant la syntaxe -v.
2. Vérifiez que le montage est correct.
3. Créez un fichier dans le répertoire monté.
4. Vérifiez la persistance.

10.3 Manipulation 9 — Montage avec --mount

1. Montez un autre volume en utilisant cette fois la syntaxe --mount.
2. Vérifiez le fonctionnement.
3. Comparez la structure de la commande avec celle utilisant -v.

Vous devez comparer :

- La lisibilité
- La précision
- La facilité de compréhension
- L'usage recommandé en contexte professionnel

Questions :

1. Quelle syntaxe vous semble la plus claire ? Pourquoi ?
2. Dans quel cas --mount peut-il être préférable ?
3. Pourquoi faut-il savoir lire et écrire les deux syntaxes ?

11. Pré-remplissage et comportement particulier des volumes

11.1 Objectif

Observer ce qui se passe lorsqu'un volume est monté sur un répertoire déjà peuplé dans l'image.

11.2 Explication du principe de Pré-remplissage automatique d'un volume

Idée générale

Quand on monte un volume Docker nommé sur un répertoire d'un conteneur, deux cas peuvent se produire :

Cas 1 — le répertoire cible dans l'image est vide

Le volume est simplement monté, sans effet particulier.

Cas 2 — le répertoire cible dans l'image contient déjà des fichiers

Docker peut initialiser automatiquement le volume avec le contenu déjà présent dans l'image, au moment du premier montage du volume. C'est ce qu'on appelle souvent le pré-remplissage automatique du volume.

Pourquoi ce comportement existe :

Docker considère qu'un volume nouvellement créé et monté sur un répertoire déjà peuplé peut devoir récupérer ce contenu initial. L'objectif est de permettre à un conteneur de fournir un contenu par défaut dans un volume. Par exemple :

- Fichiers HTML par défaut de nginx
- Fichiers d'initialisation d'une application
- Répertoires contenant une configuration de base

11.3 Manipulation 10 — Volume sur un répertoire déjà initialisé

Certaines images Docker contiennent déjà des fichiers dans des répertoires applicatifs précis. C'est le cas, par exemple, de l'image nginx, qui contient généralement des fichiers par défaut dans le répertoire : `/usr/share/nginx/html`

Lorsqu'un volume Docker vide est monté pour la première fois sur un tel répertoire, Docker peut recopier dans ce volume le contenu initial déjà présent dans l'image. Ce comportement peut surprendre si l'on pense qu'un volume monté est nécessairement vide ou qu'il remplace toujours immédiatement tout contenu existant sans autre effet.

Travail demandé :

1. Lancez un conteneur nginx sans monter de volume.
2. Accédez à l'intérieur du conteneur.
3. Observez le contenu du répertoire : `/usr/share/nginx/html`
4. Relevez les fichiers présents et vérifiez qu'ils sont déjà fournis par l'image.
5. Créez un volume Docker nommé.
6. Vérifiez son existence dans la liste des volumes disponibles.
7. Lancez un nouveau conteneur nginx.
8. Montez le volume créé sur : `/usr/share/nginx/html`
9. Accédez à l'intérieur du conteneur.

10. Observez le contenu visible dans ce répertoire.
11. Comparez ce contenu avec celui observé précédemment sans volume.
12. Ajoutez un nouveau fichier dans le répertoire monté.
13. Modifiez éventuellement un fichier déjà présent.
14. Vérifiez la présence de ces modifications.
15. Supprimez ou arrêtez le conteneur précédent.
16. Lancez un nouveau conteneur en montant le même volume sur le même chemin.
17. Vérifiez que les fichiers modifiés ou ajoutés sont toujours présents.
18. Comparez ce contenu avec le contenu initial de l'image.

À partir des manipulations réalisées, vous devez expliquer :

1. Pourquoi le volume ne semblait pas vide lors du premier montage
2. Quel contenu provenait initialement de l'image
3. Pourquoi ce contenu est ensuite devenu le contenu du volume
4. Pourquoi les modifications persistent lors des montages suivants
5. Pourquoi il ne faut pas confondre : Contenu de l'image, contenu visible dans le conteneur et contenu persistant du volume.

Questions de réflexion :

1. Pourquoi le volume peut-il contenir des fichiers dès son premier montage alors qu'il vient d'être créé ?
2. Le contenu observé après le premier montage provient-il directement de l'image ou du volume ?
3. Que se passe-t-il si le même volume est réutilisé dans un second conteneur ?
4. Le contenu initial de l'image est-il modifié lorsque l'on change les fichiers dans le volume ?
5. Pourquoi ce comportement peut-il être utile dans certains cas ?
6. Pourquoi peut-il aussi être source de confusion si l'on ne comprend pas bien le mécanisme de montage ?

12. Volume nommé vs bind mount

12.1 Objectif

Distinguer clairement deux mécanismes de stockage souvent confondus : les volumes Docker nommés et le bind mount. Ces deux mécanismes permettent de monter un espace de stockage dans un conteneur, mais ils ne répondent pas exactement aux mêmes besoins et n'offrent pas les mêmes garanties en matière :

- De persistance
- De portabilité

- D'administration
- De dépendance vis-à-vis de la machine hôte
- D'usage en développement ou en production

12.2 Manipulation 11 — Utilisation d'un bind mount

Dans Docker, il est souvent nécessaire de rendre des données accessibles à un conteneur de manière durable ou contrôlée. Pour cela, plusieurs solutions existent.

Dans les parties précédentes, vous avez étudié les volumes nommés, qui sont gérés directement par Docker.

Dans cette partie, vous allez les comparer avec les bind mounts, qui consistent à monter directement un répertoire ou un fichier de la machine hôte dans le conteneur.

Cette distinction est très importante en pratique :

- Le volume nommé est souvent utilisé pour les données persistantes applicatives
- Le bind mount est souvent utilisé en développement pour partager directement du code source ou des fichiers de configuration entre l'hôte et le conteneur

Travail demandé

A. Étude du bind mount

1. Créez un répertoire de test sur la machine hôte.
2. Vérifiez qu'il est initialement vide ou qu'il contient un contenu simple identifiable.
3. Lancez un conteneur léger.
4. Montez le répertoire de l'hôte dans un répertoire du conteneur.
5. Accédez à l'intérieur du conteneur.
6. Vérifiez la présence du contenu du répertoire monté.
7. Depuis le conteneur, créez un fichier dans le répertoire monté.
8. Quittez le conteneur.
9. Vérifiez depuis l'hôte que le fichier est bien apparu dans le répertoire local.
10. Modifiez le fichier depuis la machine hôte, ou créez-en un nouveau.
11. Relancez un conteneur avec le même bind mount.
12. Vérifiez que le conteneur voit immédiatement les modifications apportées depuis l'hôte.

B. Comparaison avec un volume nommé

13. Créez un volume Docker nommé.
14. Montez ce volume dans un conteneur.
15. Créez un fichier dans le répertoire correspondant.

16. Inspectez le volume.
17. Comparez son mode de gestion avec celui du bind mount.
18. Identifiez ce qui dépend directement de Docker et ce qui dépend directement du système de fichiers de l'hôte.

Analyse : À partir des manipulations réalisées, vous devez expliquer :

1. Ce qu'est un bind mount.
2. La différence pratique entre un volume nommé et un bind mount.
3. Pourquoi le bind mount est adapté au partage direct de fichiers en développement.
4. Pourquoi le volume nommé est mieux adapté à la persistance des données applicatives.

Questions

1. Pourquoi les modifications faites sur l'hôte sont-elles visibles immédiatement dans un **bind mount** ?
2. Pourquoi le bind mount est-il souvent utile en développement et plus pratique pour monter un dossier de code source ?
3. Pourquoi un volume nommé est-il souvent préférable pour des données applicatives persistantes ?
4. Quels risques présente un bind mount mal maîtrisé ?

12.3 Travail à faire — Tableau comparatif

Dans le rapport, vous devez produire un tableau comparatif entre :

- stockage interne au conteneur
- volume Docker nommé
- bind mount

Le tableau doit au minimum comporter les critères suivants :

- persistance
- dépendance au conteneur
- dépendance à l'hôte
- facilité d'administration
- portabilité
- usage recommandé

13. Vérification, diagnostic et gestion des volumes Docker

13.1. Objectif de cette partie

Cette partie a pour objectif d'amener l'étudiant à adopter une démarche plus rigoureuse d'analyse et de diagnostic des volumes Docker. A ce stade du TP, les étudiants ont déjà appris à :

- Créer des volumes.
- Les monter dans des conteneurs.
- Observer la persistance.
- Comparer volume nommé et bind mount.

Dans cette nouvelle partie, il s'agit d'aller plus loin en apprenant à :

- Vérifier qu'un volume est réellement utilisé
- Relier un volume à un service précis
- Distinguer un volume utile d'un volume inutile
- Détecter une erreur de montage
- Diagnostiquer une perte apparente de données
- Raisonner sur la qualité d'une configuration de persistance

13.2. Manipulation 12 — Cartographier les volumes utilisés dans l'environnement Docker

Travail demandé :

1. Listez tous les volumes présents sur votre machine Docker.
2. Identifiez ceux qui ont été créés pendant le TP.
3. Pour chacun, essayez de déterminer :
 - Quel conteneur l'utilise.
 - À quel chemin il est monté.
 - À quel usage il correspond.
4. Présentez les résultats sous forme d'un tableau de synthèse.

Objectif de la manipulation :

- Apprendre à relier un volume à un service réel et à comprendre son rôle dans l'architecture.

Remarques à relever :

- Un volume peut exister sans être immédiatement identifiable si l'on ne remonte pas jusqu'au conteneur qui l'utilise
- Le nom du volume seul ne suffit pas toujours à comprendre sa fonction
- L'inspection des volumes et des conteneurs est nécessaire pour établir une correspondance correcte

13.3. Manipulation 13 — Vérifier qu'un volume contient réellement les données attendues

Travail demandé :

1. Choisissez un volume déjà utilisé dans une manipulation précédente.
2. Montez ce volume dans un conteneur léger.
3. Parcourez le contenu du répertoire monté.
4. Vérifiez si les fichiers attendus sont présents.
5. Comparez ce contenu avec ce que vous pensiez avoir persisté.

Objectif de la manipulation :

Apprendre à vérifier directement le contenu d'un volume au lieu de supposer que les données y sont correctement stockées.

Remarques à relever :

- La présence du volume ne prouve pas automatiquement que les bonnes données y sont stockées
- Un volume peut exister tout en étant vide ou en contenant un contenu inattendu
- La vérification directe est utile lorsqu'un service semble avoir "perdu" ses données

13.3. Manipulation 14 — Identifier un montage correct et un montage inutile

Travail demandé :

1. Reprenez un conteneur déjà utilisé dans le TP.
2. Identifiez le répertoire réellement utilisé par l'application ou le service pour écrire ses données.
3. Vérifiez le chemin exact sur lequel le volume est monté.
4. Déterminez si ce montage protège réellement les données utiles.
5. Rédigez une courte conclusion :
 - Montage utile
 - Ou montage inutile / mal placé

Objectif de la manipulation :

Faire comprendre qu'un volume n'est utile que s'il est monté sur le bon répertoire.

Remarques à relever :

- Un volume peut être techniquement correct mais fonctionnellement inutile
- La persistance dépend du lien entre : Le volume, le point de montage et le chemin réellement utilisé par le service
- La bonne question n'est pas seulement "y a-t-il un volume ?" mais "ce volume protège-t-il les bonnes données ?"

13.4. Manipulation 15 — Détecter une perte apparente de données

Travail demandé :

On considère le scénario suivant : un utilisateur affirme avoir utilisé un volume Docker, mais après recreation du conteneur, il ne retrouve pas ses données. À partir de ce scénario, vous devez vérifier successivement :

1. Si le volume existe réellement
2. S’il est bien monté dans le conteneur
3. Si le point de montage est correct
4. Si le service écrit réellement dans ce répertoire
5. Si le même volume a bien été réutilisé

Puis vous devez proposer l’explication la plus probable du problème.

Objectif de la manipulation :

Construire une logique de diagnostic simple et réutilisable.

Remarques à relever :

- Une “perte de données” peut venir d’un mauvais montage et non de la disparition réelle du volume
- Il faut distinguer :
 - Volume absent
 - Volume présent mais vide
 - Volume présent mais mal monté
 - Volume correct mais non réutilisé
- Le diagnostic doit être fondé sur des vérifications et non sur une supposition

13.5. Manipulation 16 — Identifier les volumes devenus inutiles

Travail demandé :

1. Listez les volumes présents à la fin de vos manipulations.
2. Distinguez :
 - Les volumes encore utiles
 - Les volumes de test devenus inutiles
3. Justifiez votre classement.
4. Supprimez uniquement les volumes inutiles.
5. Vérifiez le résultat.

Objectif de la manipulation :

Apprendre à gérer proprement les volumes dans un environnement Docker réel.

Remarques à relever :

- un volume peut continuer à exister même après suppression du conteneur
- Il ne faut pas supprimer un volume sans savoir s'il contient encore des données utiles
- Le nettoyage des données Docker fait partie des bonnes pratiques DevOps

13.6. Manipulation 17 — Construire une procédure de diagnostic courte**Travail demandé :**

À partir des mini-manipulations précédentes, rédigez une procédure courte en 4 étapes maximum pour diagnostiquer un problème de persistance avec Docker. Votre procédure doit permettre de vérifier rapidement :

- L'existence du volume
- Le point de montage
- Le répertoire réellement utilisé par le service
- La réutilisation du bon volume

Objectif de la manipulation :

Transformer les observations techniques en méthode de travail réutilisable.

Remarques à relever :

- Une bonne procédure de diagnostic doit être courte, ordonnée et vérifiable
- Il faut partir des faits observables avant de conclure à une erreur Docker
- Une méthode simple vaut mieux qu'une suite de commandes sans logique

13.7. Questions de réflexion

1. Pourquoi un volume existant ne garantit-il pas automatiquement une bonne persistance ?
2. Pourquoi faut-il toujours vérifier le chemin réellement utilisé par le service ?
3. Pourquoi la vérification directe du contenu d'un volume est-elle utile en phase de diagnostic ?
4. Quelle erreur de configuration vous semble la plus fréquente dans l'usage des volumes Docker ?

14. Travail à remettre

Vous devez rendre dans un rapport structuré :

- Les commandes utilisées
- Les observations réalisées
- Les réponses aux questions
- Les tableaux comparatifs demandés
- Les analyses des cas de debug

15. Consignes importantes

- Toutes les manipulations doivent être réellement testées.
- Chaque résultat doit être expliqué.
- Les réponses doivent être argumentées techniquement.
- Les tableaux comparatifs doivent être clairs et exploités.